

CloudFront Geographic Restrictions using EC2 Origin

To Begin with the Lab

Summary of the Lab

In this lab, an **Amazon EC2** instance was configured as a web server and used as an origin for an **Amazon CloudFront** distribution. Unlike previous labs that used **Amazon S3** as the origin, this lab demonstrated how to use an EC2-hosted website by configuring Apache HTTP Server and deploying a custom web page. After creating the CloudFront distribution, **CloudFront Geographic Restrictions** were configured to block website access from selected countries. The restrictions were tested using a geo-location testing tool and by accessing the website from India. Finally, the restrictions were removed and normal access was restored.

Understanding Geographic Restrictions in Amazon CloudFront

CloudFront Geographic Restrictions allow content owners to control which countries can access their CloudFront distributions. CloudFront determines the viewer's country using the source IP address and then either allows or blocks access based on configured rules. This feature is commonly used for licensing requirements, regulatory compliance, regional content distribution, and restricting unwanted traffic. Geographic restrictions can be configured using either an allow list or a block list.

Example:

A video streaming company owns distribution rights only in the United States and Canada. Using CloudFront Geographic Restrictions, the company allows access only from those countries and automatically blocks viewers from all other regions.

Lab Steps

Step 1 — Create and Configure the EC2 Web Server

- Create an **Amazon EC2** instance and configure name as **web-server** and enable **Allow HTTP traffic from the internet**.

EC2 > Instances > Launch an instance

Name and tags Info

Name: [Add additional tags](#)

Application and OS Images (Amazon Machine Image) Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Q Search our full catalog including 1000s of application and OS images

Recents | My AMIs | **Quick Start**

Amazon Linux | macOS | Ubuntu | Windows | Red Hat | SUSE Linux | Debian

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI
ami-0db56f446d44f2f09 (64-bit (x86), uefi-preferred) / ami-07c18f602ca3306b9 (64-bit (Arm), uefi)

Summary

Number of instances: Info

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet. Data transfer charges are not included as part of the free tier

[Cancel](#) [Launch instance](#) [Preview code](#)

- Now we will configure it as a web server so we first must access the instance.
- For this, we will use **Command Prompt** and run the following commands.
- Now type: `cd {location_of_your_security_key}`.
- Now type: `ssh -i {security_key_name} ec2-user@{public_ipv4_address_of_ec2_instance}`.

```
ec2-user@ip-172-31-5-100:~
C:\Users\Shashank>cd downloads
C:\Users\Shashank\Downloads>ssh -i June-2026.pem ec2-user@65.2.73.108
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
[ec2-user@ip-172-31-5-100 ~]$
```

- Now login as root user by typing this command: **sudo -i**.
- Use **yum install httpd** command to configure and place webpage in Apache on the EC2 instance.

```
root@ip-172-31-6-87:~$ sudo -i
[root@ip-172-31-6-87 ~]# yum install httpd
Amazon Linux 2023 Kernel Livepatch Repository                               392 kB/s | 47 kB    00:00
Last metadata expiration check: 0:00:01 ago on Fri Jun 12 09:07:49 2026.
Dependencies resolved.
=====
Package                        Architecture      Version                                Repository          Size
=====
Installing:
httpd                          x86_64            2.4.67-1.amzn2023.0.1                amazonlinux          46 k
Installing dependencies:
apr                            x86_64            1.7.5-1.amzn2023.0.4                amazonlinux          129 k
apr-util                       x86_64            1.6.3-1.amzn2023.0.2                amazonlinux          97 k
apr-util-ldap                  x86_64            1.6.3-1.amzn2023.0.2                amazonlinux          13 k
generic-logos-httpd            noarch            18.0.0-12.amzn2023.0.3              amazonlinux          19 k
httpd-core                     x86_64            2.4.67-1.amzn2023.0.1                amazonlinux          1.4 M
httpd-filesystem               noarch            2.4.67-1.amzn2023.0.1                amazonlinux          12 k
httpd-tools                    x86_64            2.4.67-1.amzn2023.0.1                amazonlinux          80 k
libbrotli                      x86_64            1.0.9-4.amzn2023.0.2                amazonlinux          315 k
mailcap                         noarch            2.1.49-3.amzn2023.0.3                amazonlinux          33 k
Installing weak dependencies:
apr-util-openssl               x86_64            1.6.3-1.amzn2023.0.2                amazonlinux          15 k
mod_http2                      x86_64            2.0.39-1.amzn2023.0.1                amazonlinux          167 k
mod_lua                        x86_64            2.4.67-1.amzn2023.0.1                amazonlinux          59 k
=====
Transaction Summary
=====
Install 13 Packages

Total download size: 2.4 M
```

- Replace the default web page with the provided index.html content with the help of vi editor.

```
Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
httpd-core-2.4.67-1.amzn2023.0.1.x86_64
httpd-filesystem-2.4.67-1.amzn2023.0.1.noarch
httpd-tools-2.4.67-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mailcap-2.1.49-3.amzn2023.0.3.noarch
mod_lua-2.4.67-1.amzn2023.0.1.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
httpd-2.4.67-1.amzn2023.0.1.x86_64
httpd-2.4.67-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mod_http2-2.0.39-1.amzn2023.0.1.x86_64

Completed!
[root@ip-172-31-5-100 ~]# cd /var/www/html/
[root@ip-172-31-5-100 html]# vi index.html
```

- Use the above commands to change content of the default index.html and paste your code like this or use the code below.

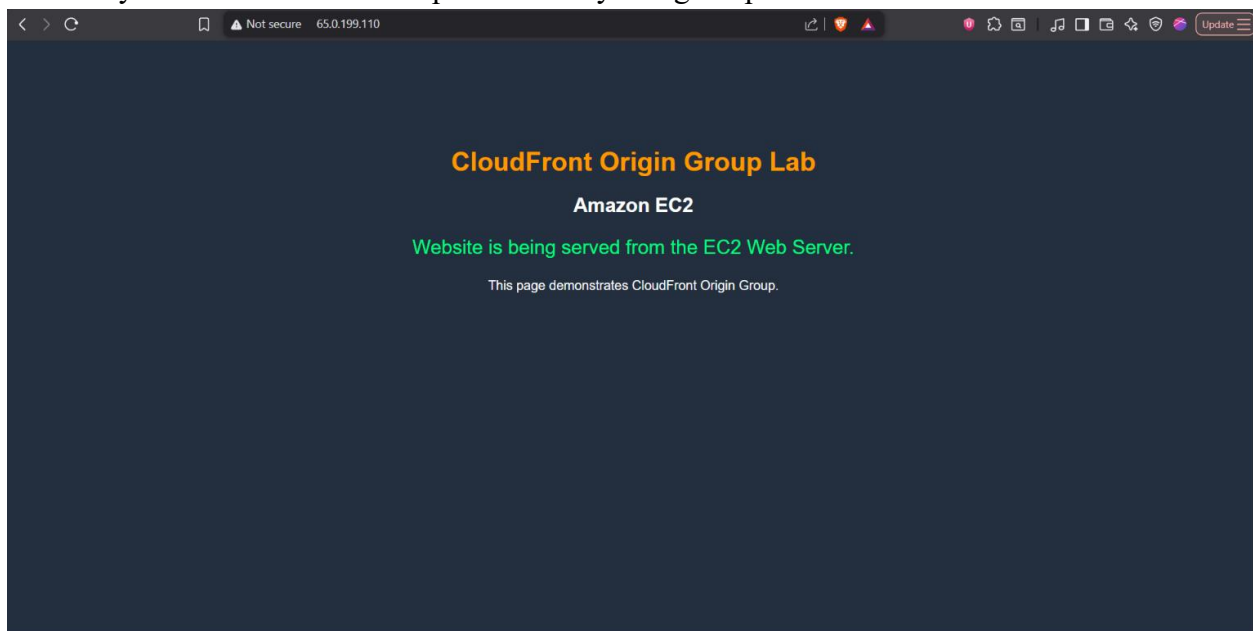
```
<!DOCTYPE html>
<html>
<head>
  <title>CloudFront Origin Group Demo</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #232F3E;
      color: white;
      text-align: center;
      padding-top: 100px;
    }
    .container {
      width: 70%;
```

```
        margin: auto;
    }
    h1 {
        color: #FF9900;
    }
    .status {
        font-size: 24px;
        color: #00FF7F;
    }
}
</style>
</head>
<body>
    <div class="container">
        <h1>CloudFront Origin Group Lab</h1>
        <h2>Amazon EC2</h2>
        <p class="status">Website is being served from the EC2 Web Server.</p>
        <p>This page demonstrates CloudFront Origin Group.</p>
    </div>
</body>
</html>
```

- Now enable and run the web server services using these commands

```
systemctl start httpd
systemctl enable httpd
```

- Verify that the EC2 website opens correctly using the public IP.



Step 2 — Create the CloudFront Distribution

- Navigate to **Amazon CloudFront**.
- Click **Create Distribution**.
- Copy the EC2 instance **Public IPv4 DNS** value.

The screenshot shows the AWS Management Console for the 'Instances' page. The instance 'web-server' (ID: i-045bba5cb4b047dd9) is selected and highlighted with a red box. The instance details page is open, showing the 'Public DNS' as 'ec2-65-0-199-110.ap-south-1.compute.amazonaws.com', which is also highlighted with a red box.

- Paste the DNS name into the **Origin Domain** field after selecting type as **Other** and create the distribution.

The screenshot shows the 'Specify origin' page in the AWS CloudFront console. The 'Other' origin type is selected, and the 'Custom origin' field is filled with the public DNS name 'ec2-65-0-199-110.ap-south-1.compute.amazonaws.com'. The 'Origin path' field is optional and set to '/path'.

- Edit the Origin.

cloudfreeks-web Standard View metrics

Details

Distribution domain name: d6lezt42n3jc6.cloudfront.net

Billing: Pay-as-you-go Switch to a plan

ARN: arn:aws:cloudfront:463646775279:distribution/ELHEUWVXHGPQA

Last modified: Deploying

General | Security | **Origins** | Behaviors | Error pages | Invalidations | Logging | Tags

Origins (1/1) Edit Delete Create origin

Filter origins by property or value

Origin name	Origin domain	Origin path	Origin type	Origin Shield region	Origin access
ec2-65-0-199-110.ap-south-1.compute.amazonaws.com	ec2-65-0-199-110.ap-south-1.compute.amazonaws.com		EC2	-	-

Origin groups (0) Edit Delete Create origin group

Filter origin groups by property or value

Origin group name | Origins | Failover criteria

No origin groups
You don't have any origin groups.

- Configure the origin protocol as **HTTP Only** and **Port** as **80**.

Edit origin

Settings

Origin domain
Choose an AWS origin, or enter your origin's domain name. [Learn more](#)

ec2-65-0-199-110.ap-south-1.compute.amazonaws.com

Enter a valid DNS domain name, such as an S3 bucket, HTTP server, or VPC origin ID.

Protocol Info

☒ HTTP only

☐ HTTPS only

☐ Match viewer

HTTP port
Enter your origin's HTTP port. The default is port 80.

80

Origin path - optional
Enter a URL path to append to the origin domain name for origin requests.

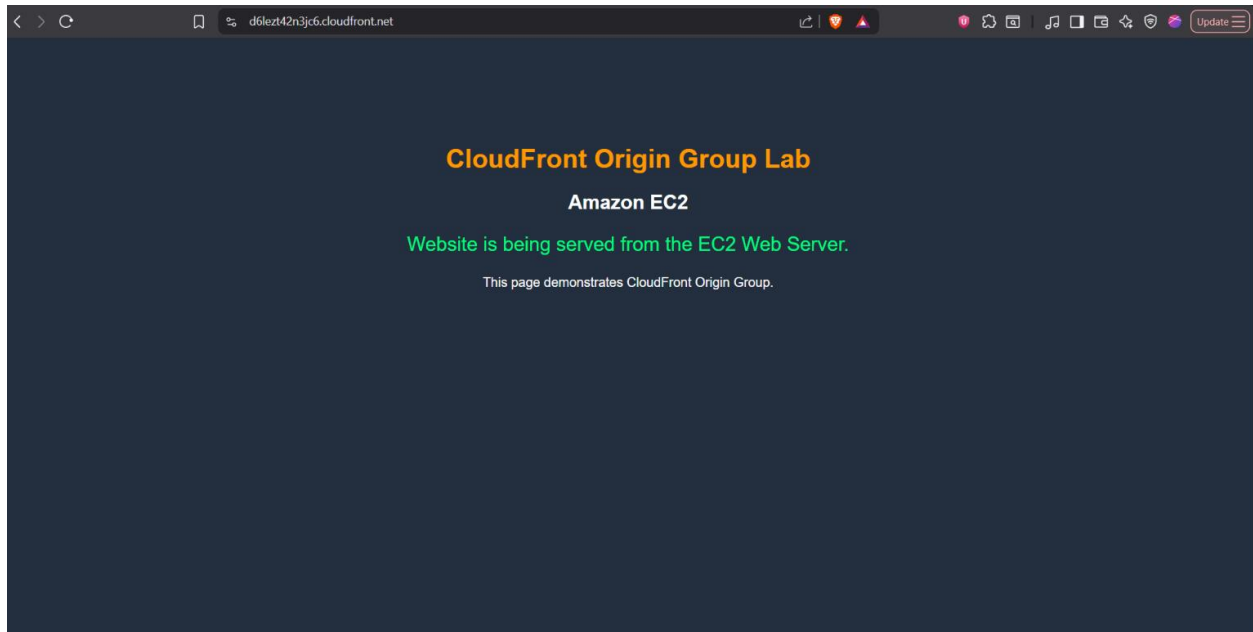
Enter the origin path

Name
Enter a name for this origin.

ec2-65-0-199-110.ap-south-1.compute.amazonaws.com-mqapowsywa3

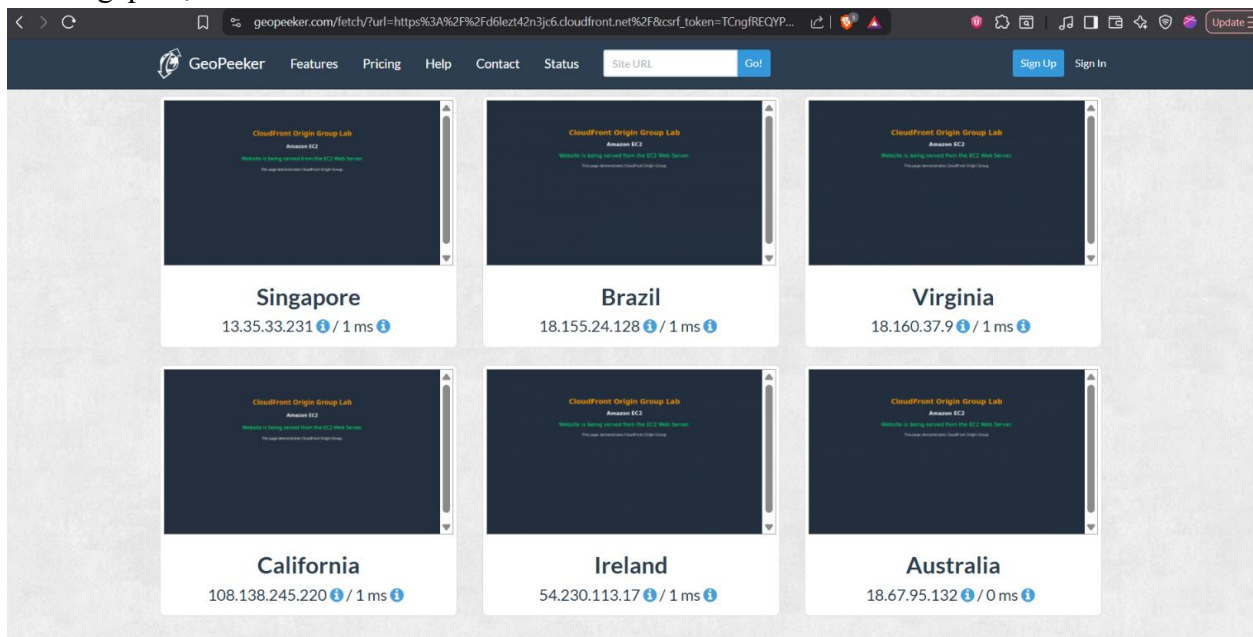
☐ **Enable origin mutual TLS**
Securely validate the connection between your origin and the CloudFront distribution.

- Wait for the distribution deployment to complete.
- Verify the website opens successfully through the CloudFront URL.



Step 3 — Test Global Website Accessibility

- Copy the CloudFront distribution URL.
- Open the website through the CloudFront URL.
- Use a geographic testing tool such as GeoPecker to test access from multiple countries.
- Verify that the website is accessible from different regions including Australia, Ireland, Singapore, and Brazil.

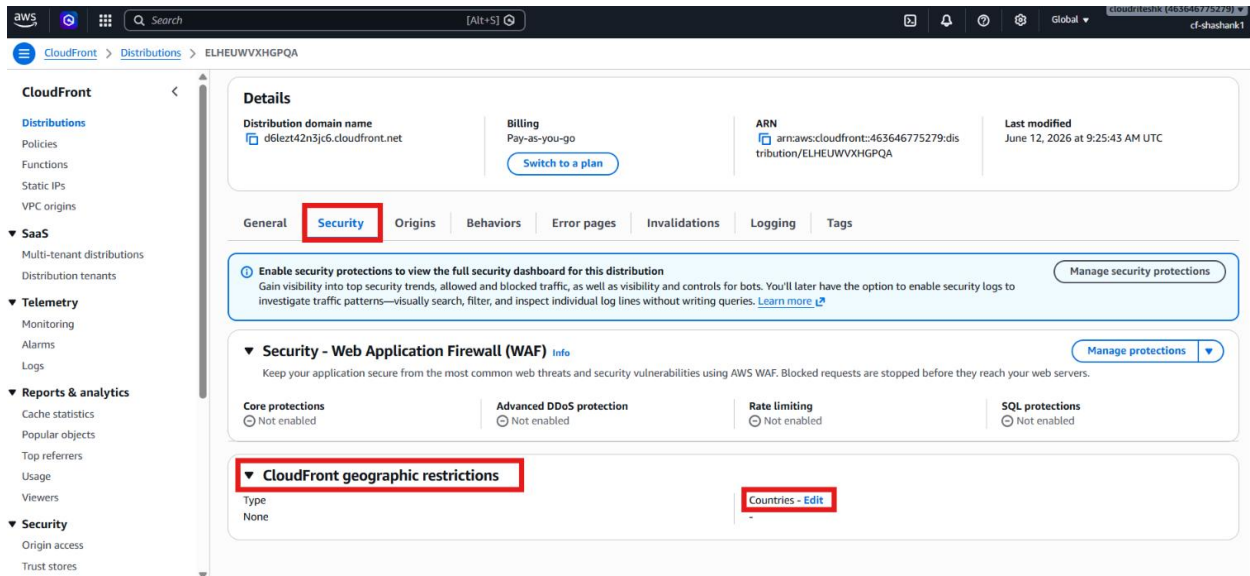


- Confirm that no geographic restrictions are currently configured.

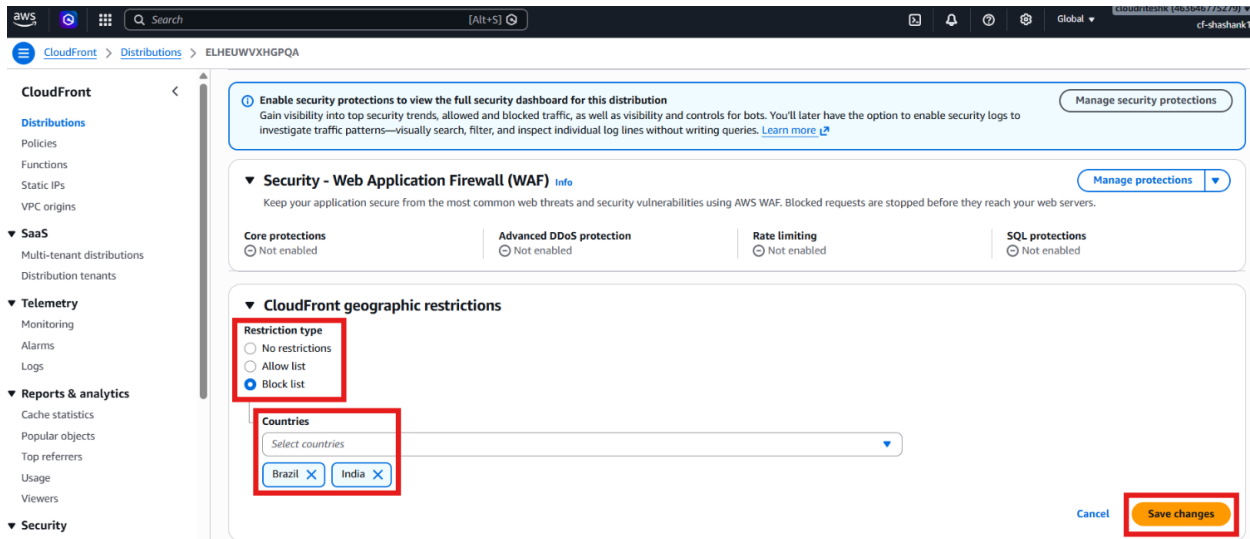
Step 4 — Configure Geographic Restrictions

Configuration Steps

- Open the CloudFront distribution.
- Navigate to **Security**.
- Locate **Geographic Restrictions**.
- Select **Countries** - **Edit**.



- Add **India**.
- Add **Brazil**.
- Save the configuration.

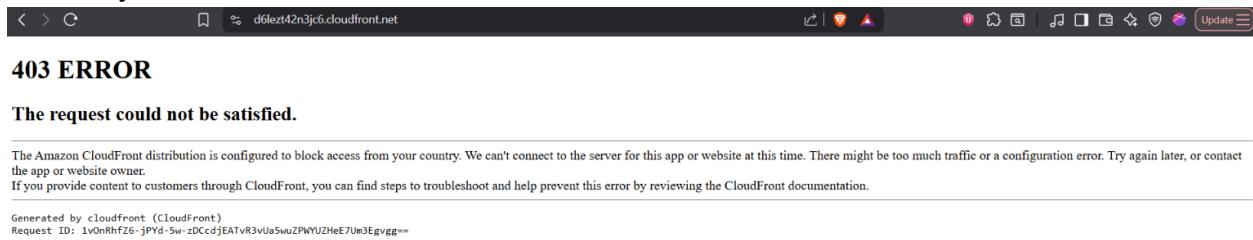


- Wait for CloudFront to redeploy the distribution.

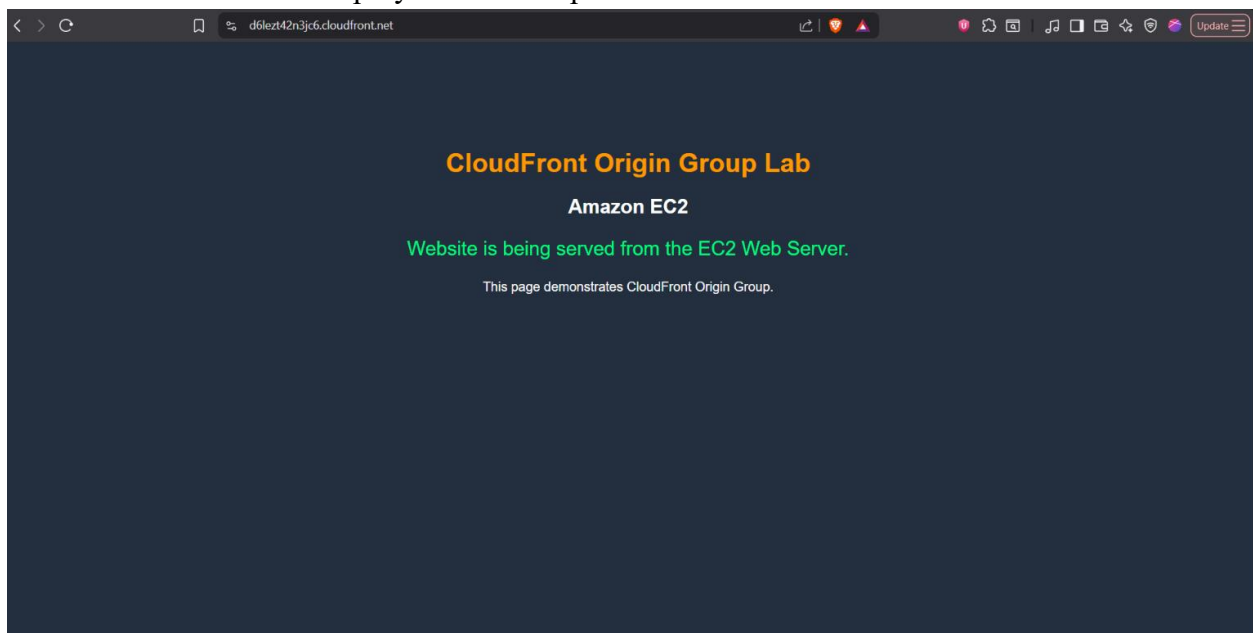
Step 5 — Verify and Remove Geographic Restrictions

- Open the CloudFront URL from India.

- Verify that CloudFront returns a **403 Forbidden** error.



- Confirm that the access denied message indicates the distribution is configured to block access from the country.
- Use GeoPeeker to verify that Brazil is also blocked.
- Confirm that countries not included in the block list can still access the website.
- Return to **CloudFront** → **Security** → **Edit**.
- Change Geographic Restrictions to **None**.
- Save the changes.
- Wait for CloudFront deployment to complete.



- Verify that the website becomes accessible again from India and Brazil.

Verification

- EC2 instance successfully hosts the website using Apache HTTP Server.
- CloudFront distribution successfully uses the EC2 instance as its origin.
- Website is accessible globally before restrictions are applied.
- Geographic restrictions successfully block access from India and Brazil.
- CloudFront returns a 403 error for blocked countries.
- Removing the restriction restores normal website access.
- Geographic filtering is successfully implemented using **Amazon CloudFront Geographic Restrictions**.